
PERSONAL & PROFESSIONAL BRIEF

Md. Jahid Hasan Raihan

Software Engineer — Dhaka, Bangladesh

DOCUMENT VERSION

1.0

STATUS

Final year · Graduating December 2026

CORE STACK

Python · Django · DRF · React · SQL

DATE

12 September 2026

SEEKING

Junior / backend / full-stack roles & internships

COMPANION DOCUMENT

PORTFOLIO_PRD.pdf

Purpose of this document. Everything on record about who Jahid is, where he came from technically, what he can actually do, and what he is working toward. It is the background half of the portfolio project — the PRD covers the *site*, this covers the *person*.

Sourcing. Every statement here came directly from Jahid during a structured discovery interview, or from work verified in his repositories. Nothing is inferred or embellished. Where something is unknown, it says so.

01 At a Glance

FIELD	VALUE
Full name	Md. Jahid Hasan Raihan
Professional title	Software Engineer
Location	Dhaka, Bangladesh
Email	jahidhr05@gmail.com
GitHub	github.com/jhraihan
LinkedIn	linkedin.com/in/md-jahid-hasan-raihan-745023393
LeetCode	leetcode.com/u/Jahid_Hasan_Raihan/ — ~150 problems solved
Degree	BSc in Computer Science and Engineering
Institution	Independent University, Bangladesh
Graduating	December 2026 — currently final year
Availability	Open to remote, hybrid and onsite roles
Geography	Bangladesh and international

THE ONE-LINE VERSION

A final-year CSE student in Dhaka who has built **five complete, production-shaped web applications** — each with authentication, role-based access control, a relational schema, and a decoupled React frontend talking to a Django REST API. Backend-leaning, self-taught beyond coursework, and deliberately honest about what he does and does not yet know.

1.1 What makes the profile unusual

The five projects are not five tutorials. They are five instances of the same hard thing: a multi-actor system where different users see different views of the same data with different rights over it. E-commerce (three roles), learning management (three roles), hospital management (four roles), plus two applications built around external model APIs with the latency and credential-handling problems that brings.

The repetition is the point. One such project could be luck or a followed guide; five is a demonstrated competency. That is the single strongest claim available from this background, and it is entirely supportable in an interview.

02 Background & Journey

2.1 How programming started

Jahid did not code before university. Programming began with the CSE degree in 2022. He chose Computer Science because the field genuinely interested him, and coding turned out to be the part that held his attention — not a career calculation made in advance.

Most of what he knows beyond coursework is **self-taught**, primarily through online courses and by building things repeatedly until they worked. This matters for interpreting the skill levels in §04: they describe capability built by shipping, not by completing a syllabus.

2.2 The learning path, in order

Python → DSA & Algorithms → Django → Django REST Framework → Databases (MySQL, SQLite, PostgreSQL) → React

STAGE	WHAT HAPPENED AND WHY
Python	The first language, and the one that still feels most natural. Everything downstream was chosen partly to stay in it.
DSA & Algorithms	Taken seriously rather than treated as a course to pass. Still the practice ground he returns to for problem-solving — ~150 LeetCode problems to date.
Django	Chosen for two reasons he states plainly: Python already felt natural, and Django let him build <i>entire applications</i> rather than fragments. This is the pivot point of the whole profile.
Django REST Framework	Followed directly from wanting a real frontend. Now his strongest single skill alongside Django itself.
Databases	Studied in earnest, not picked up incidentally — MySQL, SQLite and PostgreSQL, plus schema design and normalisation.
React	The last piece. Learned to complete the decoupled architecture the DRF work implied.

2.3 Education

FIELD	VALUE
Degree	BSc in Computer Science and Engineering
Institution	Independent University, Bangladesh
Location	Dhaka, Bangladesh
Duration	2022 → December 2026 (expected)
Standing	Final year
CGPA	3.30 — recorded here, deliberately not published on the site
Thesis	None
Prior education	HSC/SSC deliberately omitted from all materials

Relevant coursework (the seven surfaced publicly):

Data Structures Algorithms Database Management Systems Operating Systems Computer Networks
Object-Oriented Programming Software Engineering

These are listed as *subjects studied*, with no proficiency claim attached. The distinction is intentional — a course attended is not a skill demonstrated, and conflating the two is exactly the kind of overclaim the whole project avoids.

2.4 What he has not done

STATED PLAINLY, BECAUSE IT WILL BE ASKED

No professional experience. No jobs, no internships, no freelance or client work. **No certifications, competitions or awards.** Both confirmed explicitly.

This is normal for a final-year student and is presented without apology. The portfolio carries its weight through five complete systems instead. No Experience or Achievements section exists on the site, because an empty section reads worse than an absent one and a padded one is disqualifying.

03 The Five Projects

All five are solo builds, all complete, all with public repositories. They are the substance of the profile. Full case studies — problem, approach, architecture, hardest problem, lessons — are in §05 of the PRD; this is the condensed view.

#	PROJECT	DOMAIN	ROLES	DEFINING TECHNICAL CHALLENGE
1	MicroMart	E-commerce	3	Order/payment/inventory state consistency across gateway callbacks
2	EduFlow	Learning management	3	One authoritative permission layer — object-level, not just action-level
3	IntelliChat	Streaming AI chat	—	Server-Sent Events with an explicit stream-termination protocol
4	MediDesk	Hospital management	4	Modelling prescriptions as real relations; per-module access rules
5	PromptCanvas	Text-to-image	—	Long-latency requests with the model credential kept server-side

3.1 The four strongest interview answers

Each of these is a genuine engineering problem Jahid solved and can explain unscripted. They are the highest-value material in the entire profile.

Order state machines under an unreliable gateway

MICROMART

Taking a payment is simple in isolation. Keeping order, inventory and payment state consistent is not: users abandon payments mid-flow, gateway callbacks arrive late or twice, and inventory must never be decremented for an unpaid order.

WHAT HE CONCLUDED

The interesting part of a feature is usually its failure cases. Once the order states and their legal transitions were made explicit, most of the conditional logic scattered through the views simply disappeared.

Authorisation is two questions, not one

EDUFLOW

Every endpoint must answer both *may this user perform this action* and *on which records*. Answering only the first leaves a teacher able to reach another teacher's course by changing an id in the URL.

WHAT HE CONCLUDED

The interface never decides access. The API scopes every queryset by role; the frontend hides controls purely as a convenience. Roles designed in from the start stay in one place — roles added afterwards become scattered conditionals.

Streaming breaks the request/response shape

INTELLICHAT

The connection stays open while text arrives in fragments, and a stream can end in more than one way: complete, fail midway, or the client disconnects during generation. He defined an explicit event protocol — `meta`, repeated `delta`, then `done` or `error` — so the frontend always knows which occurred rather than inferring it from a stream that simply stopped.

WHAT HE CONCLUDED

Designing the client/server protocol first made both sides straightforward. And persistence changes what an interface can promise: once conversations were stored rather than held in memory, a reload stopped being a data-loss event.

The schema decides what is possible later

MEDIDESK

Storing a prescription as text would have been quick, but a prescription is genuinely structured: one consultation links to several medicines, each with its own dosage and duration.

WHAT HE CONCLUDED

Because prescriptions were modelled as real relations, catalogue search and dosage linking came almost for free; a text field would have required a migration and a rewrite. An appointment is likewise a lifecycle, not a row.

04 Technical Skills

Levels are **named, never numeric**. A percentage bar implies a precision that does not exist and invites a comparison that cannot be won. Ordering: **Core** → **Strong** → **Working Knowledge** → **Familiar**.

CATEGORY	CORE	STRONG	WORKING	FAMILIAR
Languages	Python	SQL, HTML, CSS	JavaScript	Java
Backend	Django, Django REST Framework, Django ORM	REST API design, Authentication (JWT / session / OAuth)	—	FastAPI, Node.js
Frontend	—	React, React Router, Fetch API	Tailwind CSS, Context API, Vite, Bootstrap	Next.js
Databases	—	MySQL, PostgreSQL, SQLite, Design & normalisation	Query optimisation	—
DevOps	—	Git, GitHub, Environment management	Linux, Vercel / Netlify, Railway / Render / PythonAnywhere	Nginx, GitHub Actions, AWS
Tools	—	VS Code, Postman, Git CLI, AI coding tools	pgAdmin / MySQL Workbench	Figma

The featured five, surfaced most prominently: **Django** **Django REST Framework** **React** **MySQL** **Python**

4.1 How to read these levels

LEVEL	WHAT IT IS CLAIMING
Core	Used across multiple complete projects; can build unaided and explain the internals. Django, DRF and Python only.
Strong	Comfortable and productive in real projects, with a working grasp of the trade-offs.
Working knowledge	Has used it for real work and can be productive, but would not claim depth.
Familiar	Has touched it and understands the shape of it. Explicitly <i>not</i> a claim of proficiency.

WHY THIS MATTERS

The levels are set to survive questioning. Nothing sits a level higher than it can be defended at, which means every claim on the site is one he can be pushed on comfortably — an advantage that inflated self-assessment gives up.

4.2 Where AI fits — stated precisely

AN IMPORTANT DISTINCTION

Two projects (IntelliChat, PromptCanvas) integrate external model APIs — Google Gemini and Hugging Face. That is **software engineering work against an HTTP API**: streaming, latency, credential handling, persistence.

It is **not** a claim of AI/ML expertise, and Jahid asked explicitly that it never be presented as one. There is no AI/ML skills section anywhere — no RAG, embeddings, vector databases, LangChain, agents or fine-tuning. He is at the learning stage and said so; even a "currently learning AI engineering" line was drafted and then removed at his request.

His stated long-term ambition is to become an **AI-native software engineer** — building systems where modern AI is a first-class part of the architecture rather than an afterthought. That is recorded here as a goal and, on his instruction, appears nowhere on the public site.

5.1 Stated engineering priorities

In his own ordering:

PRIORITY	VISIBLE IN THE WORK AS
1. Clean code	Explicit state machines replacing scattered conditionals; permission logic held in one place rather than repeated per view.
2. Performance	Querysets scoped at the API layer; interest in query optimisation; streaming rather than blocking responses.
3. Correct architecture	The decoupled React/DRF pattern applied consistently across all five projects, with the schema treated as the decision that constrains everything after it.

5.2 A consistent instinct

Across four independent projects and four independent write-ups, the same conclusion appears in different words each time: **get the structure right early, because structure decided late becomes mess everywhere.**

- *MicroMart* — "once the transitions were explicit, most of the conditional logic disappeared"
- *EduFlow* — "roles designed in from the start stay in one place; roles added afterwards become scattered conditionals"
- *MediDesk* — "the schema decides what the application can do later"
- *IntelliChat* — "designing the client/server protocol first made both sides straightforward"

Four projects converging on one principle, arrived at independently, is a stronger signal than any single claim on a CV. It is worth raising in an interview.

5.3 A second recurring theme

He is consistently drawn to the **failure cases** rather than the happy path — abandoned payments, duplicate callbacks, streams that die mid-generation, ids changed in the URL, reloads that lose state. That is unusual at this level and is the most senior-reading instinct in the profile.

5.4 Self-assessment

He chose **authenticity over looking impressive** at every point where the two conflicted, and asked that the site read confident without claiming to know everything. Every exclusion in the portfolio — no fake experience, no certifications, no AI expertise claim, no skill percentages, no displayed CGPA — was his decision, not a limitation imposed on him.

06 What He Is Looking For

DIMENSION	TARGET
Roles	Junior Software Engineer · Backend Developer · Full-Stack Developer · Django Developer · Internship
Geography	Bangladesh and international
Work mode	No preference — remote, hybrid or onsite
Company type	No preference — startup, established company or agency
Industry	No preference
Available from	Graduating December 2026

6.1 Where he fits best

The evidence points at **backend or full-stack work in a Python/Django shop**. He is strongest where there is a real data model, multiple kinds of user, and permissions that actually matter — which is most internal tooling, most B2B software, and most platform work.

He is weakest, by his own accounting, on production operations: deployment, CI/CD and cloud infrastructure sit at working or familiar, and nothing of his is deployed yet. A team with existing infrastructure and code review is a better first environment than one where he would own the platform alone.

6.2 How he wants to come across

IN HIS OWN WORDS — FIRST IMPRESSION

"This is a serious developer who can build real-world full-stack applications and is constantly pushing himself to learn and improve."

IN HIS OWN WORDS — WHAT SHOULD BE REMEMBERED

"A hands-on full-stack developer who builds practical projects with Django and React, understands how the pieces fit together, and has the curiosity and persistence to keep getting better."

Tone he asked for: technical · professional · confident · friendly.

6.3 Five things he explicitly did not want

AVOID	WHY IT MATTERS TO HIM
Arrogance	No "expert", no "master", no superlatives anywhere.
"Just a student"	His highest-priority guard. The degree is a credential, not a qualifier — no "aspiring", no apologetic framing. The title is <i>Software Engineer</i> .
Generic template	No purple gradients, no skill bars, no waving-hand greeting.
Over-designed	No particles, cursor trails or entry animations.
Unfinished	No broken links, no "coming soon", no placeholder text.

07 Honest Assessment

Written for whoever picks this up later — including Jahid himself. This is the section that makes the document useful rather than flattering.

7.1 Genuine strengths

STRENGTH	EVIDENCE
Complete systems, repeatedly	Five full applications, not five features. The consistency is the argument.
Authorisation modelling	Three-, three- and four-role systems with object-level scoping enforced at the API. Better understood than is typical pre-graduation.
Relational schema design	The MediDesk prescription decision is a real design judgement, correctly made and correctly explained.
Failure-case thinking	Gateway callbacks, stream termination, abandoned payments. The most senior-reading instinct he has.
Calibrated self-assessment	Rare and undervalued. Everything claimed survives questioning.

7.2 Real gaps — stated, not hidden

GAP	CURRENT POSITION
No production deployment	Nothing of his is live. The highest-value gap to close, and the cheapest — deploying one project would materially strengthen the profile.
No professional experience	No jobs or internships. Only time and a first role fix this; the projects are the substitute.
No team collaboration	All five projects are solo. No pull request review, no shared codebase, no merge conflicts. Worth naming honestly if asked.
Ops and CI/CD	Working-to-familiar. Nginx, GitHub Actions and AWS are familiar-level, and untested in production.
Frontend depth	React is Strong, not Core. Deliberate — he is a backend engineer who can build a competent frontend, and presents himself that way.
Testing on the five projects	The portfolio site itself has 55 tests. Whether the five showcased projects have comparable coverage was never established — likely to be asked.

7.3 The highest-leverage next steps

1. **Deploy one project — ideally MicroMart.** A live URL alongside the case study converts the strongest narrative into something a reviewer can click. Single biggest return available.
2. **Record the walkthrough videos.** Already the chosen plan. A recruiter who watches two minutes of a real system is past the credential question entirely.
3. **Finish the repository cleanup.** The five repos are being reorganised and one URL is a known mismatch. Repos are checked more than portfolios are read.
4. **Add screenshots.** Placeholders are designed rather than broken, but five real interfaces would do a great deal of work.
5. **Consider one collaborative contribution.** Even a modest merged open-source PR would answer the only structural gap the projects cannot address alone.

7.4 Questions he should expect

- *"Walk me through what happens when a payment callback arrives twice."* — He can answer this well. It is his best

- *"How do you stop one teacher reading another teacher's course?"* — Also strong; he has the object-level answer ready.
- *"Is any of this deployed?"* — Currently no. Worth answering directly, with the walkthrough videos offered.
- *"Have you worked in a team?"* — No. Best answered plainly rather than stretched.
- *"How much AI/ML do you actually know?"* — He should give the §4.2 answer verbatim: integration work, not model expertise. It is a credibility win, not a weakness.
- *"Do these projects have tests?"* — Unestablished. Worth checking before interviewing.

08 Notes for Future Contributors

If you are picking this project up — including Jahid returning after a gap — these are the constraints that are **binding**, not stylistic preferences.

RULE ONE

Never invent anything. No jobs, companies, internships, clients, awards, certifications, user counts, revenue, statistics, years of experience, project features, technologies, achievements or testimonials. Where information is missing, mark it [CONTENT NEEDED] and ask. **Authenticity outranks impressiveness** — that was Jahid's explicit instruction, chosen knowing the cost.

STANDING EXCLUSION	REASON
No AI/ML skills section	Learning stage. No RAG, embeddings, vector DBs, LangChain, agents or fine-tuning anywhere.
No "currently learning AI" line	Drafted once, then removed at his request. Do not reintroduce it.
No Experience section	Nothing genuine to put in it.
No Achievements section	Nothing significant to display.
CGPA not displayed	Stored with <code>show_result=False</code> . Recorded privately by design.
LeetCode count not displayed	Footer link only. The number invites comparison.
No skill percentages	Named levels only. Enforced by an automated test.
Title stays "Software Engineer"	Never "student", never "aspiring".

8.1 Two facts that look like errors but are not

LOOKS WRONG	ACTUALLY CORRECT
MySQL is a featured skill, but the portfolio site runs PostgreSQL	Both are true. MySQL is his strongest database and MicroMart's; PostgreSQL is the portfolio <i>site's</i> own database, switched for hosting economics. Do not "fix" either into the other.
PromptCanvas's repo URL reads <code>tech-store-platform-django-react</code>	A genuine known issue — a renamed repo or transcription error. Flagged as outstanding, awaiting his verification. Do not guess a replacement URL.

8.2 Still outstanding from him

1. Screenshots for all five projects
2. YouTube walkthrough links
3. Résumé PDF
4. Verified repository URLs (PromptCanvas especially)
5. Deployment
6. Confirmation of which email address is the public one

Every one of these is wired to **hide rather than break** while absent, and all are content changes through the Django admin — none require code. The site is shippable now and improves as each arrives.